

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA**

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



**LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG
ĐỀ TÀI HỆ THỐNG ĐĂNG KÝ MÔN HỌC CHO SINH
VIÊN**

Giảng viên hướng dẫn: Trần Đình Nam Sơn

Sinh viên thực hiện: Bùi Khánh Linh

Trương Việt Thành

Nguyễn Việt Bách

Đào Việt Anh

Nguyễn Đức Toàn

Nguyễn Công Hoàn

Lớp tín chỉ: CSE703048-2-3-25(N01)

Hà Nội, 2026

LỜI CẢM ƠN

Để hoàn thành được đề tài này, đầu tiên nhóm chúng em xin gửi lời cảm ơn chân thành nhất đến Ban Giám hiệu Đại học Phenikaa và Trường Công nghệ thông tin Phenikaa. Nhà trường đã tạo ra một môi trường học tập hiện đại, chuyên nghiệp đồng thời tạo điều kiện thuận lợi nhất để sinh viên được tiếp cận với các kiến thức nền tảng vững chắc.

Đặc biệt, nhóm chúng em xin bày tỏ lòng biết ơn sâu sắc đến Thầy Trần Đình Nam Sơn, giảng viên hướng dẫn trực tiếp học phần Lập trình hướng đối tượng. Những bài giảng tận tâm cùng những lời góp ý, định hướng mang tính thực tiễn cao của thầy trong suốt quá trình học tập không chỉ giúp chúng em hiểu rõ về bản chất của tư duy lập trình, mà còn là kim chỉ nam để nhóm từng bước phân tích, thiết kế và hoàn thiện cấu trúc cho đề tài này.

Trong quá trình thực hiện dự án, dù đã có sự phân công công việc cụ thể và phối hợp chặt chẽ giữa các thành viên để áp dụng kiến trúc 3 tầng cũng như các nguyên lý hướng đối tượng nhưng do giới hạn về mặt thời gian và kinh nghiệm thực tiễn, báo cáo cũng như mã nguồn của hệ thống chắc chắn không tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được sự thông cảm và những ý kiến đóng góp quý báu từ thầy để đề tài được hoàn thiện hơn, đồng thời giúp chúng em rút ra những bài học kinh nghiệm quý giá cho chặng đường học tập và làm việc sau này.

Chúng em xin chân trọng cảm ơn!

Hà Nội, tháng 7 năm 2026

Sinh viên thực hiện

Nhóm 02

LỜI NÓI ĐẦU

Trong xu thế chuyển đổi số đang diễn ra mạnh mẽ tại các cơ sở giáo dục đại học, mô hình đào tạo theo hệ thống tín chỉ đã trở thành một bước tiến tất yếu nhằm tối ưu hoá nguồn lực và cá nhân hoá lộ trình học tập của sinh viên. Tuy nhiên, sự linh hoạt này cũng đặt ra bài toán lớn cho công tác quản lý của phòng đào tạo. Việc ghi nhận nguyện vọng của người học giờ đây không chỉ đơn thuần là lên danh sách, mà đòi hỏi một hệ thống có khả năng kiểm soát tự động các nghiệp vụ phức tạp, yêu cầu tính chính xác tuyệt đối.

Thực tế cho thấy, trong mỗi kỳ đăng ký môn học, sinh viên thường phải tự xoay sở với hàng loạt các quy tắc nghiệp vụ nghiêm ngặt như: giới hạn số lượng tín chỉ, kiểm soát sĩ số từng lớp học để đảm bảo chất lượng và đặc biệt là bài toán ngăn chặn sự chồng chéo về thời khoá biểu. Nếu không có sự can thiệp của một hệ thống công nghệ thông minh, việc quản lý thủ công sẽ dẫn đến nhiều sai sót, trực tiếp ảnh hưởng đến kế hoạch tốt nghiệp và quyền lợi chính đáng của người học.

Xuất phát từ những yêu cầu cấp thiết đó, cùng với nền tảng kiến thức đã được trang bị, nhóm chúng em quyết định thực hiện đề tài “Hệ thống đăng ký môn học cho sinh viên”. Đề tài hướng tới việc xây dựng một chương trình mô phỏng toàn diện dựa trên ngôn ngữ Java và áp dụng triệt để các nguyên lý trong Lập trình hướng đối tượng.

Dự án không chỉ dừng lại ở mục tiêu giải quyết một bài toán kỹ thuật thông thường mà đây còn là môi trường thực tiễn quý giá để nhóm thực hành và chuẩn hoá tư duy thiết kế phần mềm, thông qua việc ứng dụng 4 tính chất nền tảng của OOP. Đề tài là một bước đệm quan trọng giúp nhóm mô phỏng lại quy trình phát triển phần mềm toàn diện trong thực tế.

PHÂN CÔNG NHIỆM VỤ

STT	Thành viên	Việc cần làm
1	Bùi Khánh Linh-Trưởng nhóm	1. Khởi tạo project – Spring Boot project (Java 17+, Maven, Spring Web, Gson) – Cấu hình CORS – Tạo cấu trúc package: model/, service/, repository/, handler/, utils/, exception/, data/ 2. Class Diagram chính thức bằng draw.io 3. Thiết kế schema JSON cho các file – studens.json, lectures.json, courses.json, schedules.json, registration.json, logs.json. – Thiết kế đầy đủ field, kiểu dữ liệu, tạo sẵn file JSON rỗng trong data/. 4. Tạo class rỗng (skeleton) chỉ có thuộc tính private + getter/setter mà chưa có logic nghiệp vụ • User(abstract), Student, Lecturer (kế thừa User), Course, Schedule 5. Utils dùng chung – FileUtils: đọc/ghi JSON bằng Gson. – IdGenerator: sinh mã tự tăng – PasswordUtils: hash password 6. Exception hierarchy – UserNotFoundException, CourseNotFoundException, CourseFullException – DuplicateRegistrationException, CreditLimitExceededException – ScheduleConflictException, RegistrationNotFoundException 7. Khai báo interface (chưa cài đặt) – Registrable – CourseValidator 8. Code – Model: Registration, RegistrationDetail đầy đủ – RegistrationHandler – REST endpoint POST/api/ registration 9. Viết báo cáo: Chương 1,2,3,4,7

2	Trương Việt Thành	Việc cần làm: – Cài đặt interface CourseValidator – Viết 4 class: CreditLimitValidator, ScheduleConflictValidator, CapacityValidator, DuplicateRegistrationValidator – Viết CourseValidatorService – gom danh sách validator, chạy tuần tự, trả về lỗi cụ thể Điều kiện cần: Student.getMaxCredits(), Course.getMaxSlots(), Course.getRegisteredSlots(), Schedule từ giai đoạn 0 Method công khai: CourseValidatorService.validateAll(Student, Course) → trả danh sách lỗi (nếu có) Giai đoạn cuối: Viết khoảng 3 tình huống test riêng cho từng validator Viết báo cáo: Mục 5.2.3
3	Nguyễn Việt Bách	Việc cần làm: – Model: hoàn thiện logic thật cho User (abstract, login()), Student (dựa trên skeleton Giai đoạn 0) – Repository: UserRepository — findByUsername(), findById(), save() – Service: AuthService (đăng nhập, hash password), StudentService (thông tin sinh viên, tổng tín chỉ) – Handler: AuthHandler (POST /api/login), StudentHandler (GET /api/students/{id}) – Frontend: LoginPage.jsx Phụ thuộc: Dùng PasswordUtils từ Giai đoạn 0 → có thể bắt đầu ngay. Method công khai cho module khác: – StudentService.findById(String id) – StudentService.getMaxCredits(String id) Giai đoạn cuối: Viết ≥ 2 test case (đăng nhập đúng/sai password). Viết báo cáo: Mục 5.2.1

4	Đào Việt Anh	Việc cần làm: – Bổ sung method cho RegistrationRepository (tạo sẵn ở Giai đoạn 0) – Service: CancellationService (hủy đăng ký, kiểm tra đã đăng ký chưa), SummaryService (tổng tín chỉ, danh sách môn đã đăng ký) – Handler: DELETE /api/registrations/{id}, GET /api/registrations/{studentId}/summary – Frontend: MyRegistrationsPage.jsx — hiển thị môn đã đăng ký, nút hủy, tổng tín chỉ Phụ thuộc: Dùng Registration, RegistrationDetail đã code thật từ Giai đoạn 0. Method công khai: – SummaryService.getTotalCredits(String studentId) — dùng trong RegistrationService để phối hợp với CreditLimitValidator Giai đoạn cuối: Viết ≥ 2 test case (hủy hợp lệ, hủy môn chưa đăng ký → báo lỗi). Viết báo cáo: Mục 5.2.4
5	Nguyễn Đức Toàn	Việc cần làm: – Model: hoàn thiện logic thật cho Course, Schedule (dựa trên skeleton Giai đoạn 0) – Repository: CourseRepository — findAll(), findById(), search(), updateSlots() – Service: CourseService — CRUD, tìm kiếm theo mã/tên, kiểm tra còn chỗ – Handler: CourseHandler (GET /api/courses, GET /api/courses/{id}, GET /api/courses/search) – Frontend: CoursePage.jsx — danh sách + tìm kiếm Phụ thuộc: Không phụ thuộc module nào khác Method công khai cho module khác: – CourseService.findById(String id) – CourseService.hasAvailableSlot(String courseId) Giai đoạn cuối: Viết ≥ 2 test case (tìm kiếm đúng, kiểm tra còn chỗ).

6	Nguyễn Công Hoàn	Việc cần làm: – Model: hoàn thiện logic thật cho Lecturer (dựa trên skeleton Giai đoạn 0), tạo model LogEntry – Repository: LogRepository (ghi log ra file), truy vấn sinh viên theo môn của giảng viên – Service: LecturerService (xem danh sách sinh viên đăng ký môn mình dạy), LogService (ghi log đăng ký/hủy/đăng nhập) – Handler: LecturerHandler (GET /api/lecturers/{id}/students) – Frontend: LecturerPage.jsx Phụ thuộc: Không phụ thuộc module nào khác để bắt đầu, nhưng LogService.log() sẽ được tất cả module khác gọi vào → cần hoàn thành sớm trong Giai đoạn 1 để người khác kịp tích hợp. Method công khai cho module khác: – LogService.log (action, actor, result) Giai đoạn cuối: Viết ≥ 2 test case. Viết báo cáo: Mục 5.2.5
---	------------------	--

DANH SÁCH THÀNH VIÊN VÀ NHÁNH LÀM VIỆC TRÊN GITHUB

Link Repository: https://github.com/bklinh-1005/Nhom_2_OOP.git

STT	Họ và tên	Mã sinh viên	Tài khoản GitHub	Nhánh làm việc
1	Bùi Khánh Linh	24100543	bklinh-1005	main
2	Trương Việt Thành	24106898	agatha0405	Truong-Viet-Thanh
3	Nguyễn Việt Bách	24100040	24100040-afk	nguyen-viet-bach
4	Đào Việt Anh	24100228	24100228-pixel	dao-viet-anh
5	Nguyễn Đức Toàn	24100230	24100230-lab	Nguyen-Toan
6	Nguyễn Công Hoàn	24100076	Conghoan206	nguyen-cong-hoan

MỤC LỤC

LỜI CẢM ƠN	2
LỜI NÓI ĐẦU	3
PHÂN CÔNG NHIỆM VỤ	4
DANH SÁCH THÀNH VIÊN VÀ NHÁNH LÀM VIỆC TRÊN GITHUB.....	8
CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI	12
1.1 Bối cảnh.....	12
1.2 Mục tiêu dự án	13
1.2.1 Về nghiệp vụ và chức năng hệ thống	13
1.2.2 Về kỹ thuật và kiến trúc phần mềm	13
1.3 Đối tượng sử dụng	13
CHƯƠNG 2: PHÂN TÍCH YÊU CẦU HỆ THỐNG	15
2.1 Yêu cầu chức năng.....	15
2.2. Yêu cầu phi chức năng.....	15
2.3 Các quy tắc nghiệp vụ	16
2.4 Biểu đồ Use Case.....	17
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	19
3.1 Kiến trúc phần mềm.....	19
3.2 Class Diagram	19
3.3 Sequence Diagram	20
3.4 Mock Database.....	21
CHƯƠNG 4: TỔ CHỨC MÃ NGUỒN VÀ CÔNG NGHỆ.....	23
4.1 Cấu trúc Package	23
4.2 Các công nghệ và công cụ sử dụng.....	24
4.2.1 Công nghệ Backend	24
4.2.2 Công nghệ Frontend.....	25
4.2.3 Quản lý dự án và mã nguồn	25
CHƯƠNG 5: CÀI ĐẶT CHI TIẾT VÀ ÁP DỤNG TƯ DUY OOP	26
5.1 Áp dụng các nguyên lý OOP	26
5.1.1 Tính đóng gói.....	26
5.1.2 Tính kế thừa	26
5.1.3 Tính đa hình và trừu tượng.....	26
5.2 Cài đặt các Module chức năng	27
5.2.1 Module Xác thực và thông tin sinh viên.....	27

5.2.2 Module Quản lý môn học và lịch học.....	28
5.2.2.2 Các class/method chính đã viết	28
5.2.2.3 Nguyên lý OOP áp dụng trong module này.....	29
5.2.2.4 Ảnh minh họa.....	29
5.2.2.6 Kết quả test	30
5.2.3 Module Kiểm tra điều kiện đăng ký	31
5.2.3.1 Mô tả chức năng	31
5.2.3.2 Các class/method chính:	31
5.2.3.3 Nguyên lý OOP áp dụng trong module này.....	31
5.2.3.4 Ảnh minh họa.....	32
5.2.3.5 Kết quả test	33
5.2.4 Module Hủy đăng ký và tổng hợp kết quả	34
5.2.4.1 Mô tả chức năng	34
5.2.4.2 Các class/method chính đã viết	34
5.2.4.3 Nguyên lý OOP áp dụng trong module này.....	35
5.2.4.4 Ảnh minh họa.....	35
5.2.4.5 Kết quả test	36
5.2.5 Module ghi nhật ký hệ thống.....	37
5.2.5.1 Mô tả chức năng	37
5.2.5.2 Các class/method chính	37
5.2.5.3 Nguyên lý OOP áp dụng trong module này.....	37
5.2.5.4 Ảnh minh họa.....	38
5.2.5.5 Kết quả test	38
5.3 Tiện ích và Xử lý ngoại lệ	39
CHƯƠNG 6: KIỂM THỬ HỆ THỐNG	40
6.1 Môi trường và phương pháp kiểm thử.....	40
6.2 Kiểm thử các nghiệp vụ chính	41
6.2.1 Kiểm thử chức năng đăng nhập	41
6.2.2 Kiểm thử chức năng tra cứu môn học.....	41
6.2.3 Kiểm thử nghiệp vụ đăng ký môn học	42
6.2.4 Kiểm thử chức năng huỷ đăng ký.....	42
6.2.5 Kiểm thử chức năng tổng hợp kết quả đăng ký	43
6.2.6 Kiểm thử chức năng ghi nhật ký hệ thống	43
6.3 Đánh giá kết quả kiểm thử	43

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	45
7.1 Kết luận	45
7.2 Đánh giá kết quả đạt được.....	45
7.3 Hướng phát triển trong tương lai	46
TÀI LIỆU THAM KHẢO	47

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh

Trong xu thế chuyển đổi số mạnh mẽ tại các cơ sở giáo dục đại học, việc chuyển đổi từ mô hình đào tạo niên chế sang hệ thống tín chỉ đã trở thành một bước tiến tất yếu nhằm cá nhân hóa lộ trình học tập và tối ưu hóa nguồn lực đào tạo. Tuy nhiên, sự linh hoạt này cũng đặt ra những thách thức đáng kể cho công tác quản lý của phòng đào tạo. Một hệ thống đăng ký môn học hiện đại không chỉ đơn thuần là nơi ghi nhận danh sách mà còn đóng vai trò là một bộ máy kiểm soát nghiệp vụ phức tạp, đòi hỏi tính chính xác tuyệt đối trong việc xử lý các ràng buộc về điều kiện học tập.

Thực tế cho thấy, sinh viên thường gặp phải nhiều khó khăn trong việc tự xây dựng thời khóa biểu khi phải đối mặt với các quy tắc nghiệp vụ nghiêm ngặt như: giới hạn số tín chỉ tối đa được phép đăng ký trong một học kỳ, kiểm soát sĩ số tối đa của từng lớp học để đảm bảo chất lượng giảng dạy và đặc biệt là ngăn chặn tình trạng trùng lịch học giữa các môn. Nếu thiếu đi một công cụ hỗ trợ thông minh, việc quản lý thủ công hoặc hậu sẽ dễ dẫn đến sai sót, gây ảnh hưởng trực tiếp đến quyền lợi và kế hoạch tốt nghiệp của người học.

Từ nhu cầu thực tiễn đó, nhóm thực hiện đề tài “Hệ thống đăng ký môn học cho sinh viên” nhằm xây dựng một chương trình mô phỏng toàn diện dựa trên nền tảng Java OOP. Dự án thiết kế theo kiến trúc 3 tầng chuyên nghiệp, giao tiếp qua HTTP request/response và sử dụng cơ chế Mook Database bằng JSON để lưu trữ dữ liệu bền vững. Việc áp dụng kiến trúc này không chỉ giúp tách biệt rõ ràng trách nhiệm giữa Frontend và Backend mà còn đảm bảo hệ thống có khả năng mở rộng và dễ bảo trì theo tiêu chuẩn doanh nghiệp.

Dự án Hệ thống đăng ký môn học cho sinh viên không chỉ dừng lại ở việc giải quyết một bài toán kỹ thuật mà còn là môi trường để nhóm thực hiện rèn luyện tư duy thiết kế hướng đối tượng chuẩn mực thông qua các nguyên lý đóng gói, kế thừa, đa hình và trừu tượng. Thông qua việc phối hợp nhóm chặt chẽ và quản lý mã nguồn tập trung trên GitHub, đề tài hướng tới mục tiêu mô phỏng quy trình phát triển phần mềm thực tế từ khâu phân tích yêu cầu đến cài đặt các module chức năng chuyên như hệ thống xác thực, bộ lọc điều kiện đăng ký và ghi nhận ký hệ thống.

1.2 Mục tiêu dự án

Mục tiêu cốt lõi của dự án là xây dựng một hệ thống mô phỏng đăng ký môn học hoàn chỉnh, đảm bảo tính thực tiễn về mặt nghiệp vụ và sự chuẩn mực trong kỹ thuật lập trình hướng đối tượng. Cụ thể, đề tài hướng tới các mục tiêu chi tiết sau:

1.2.1 Về nghiệp vụ và chức năng hệ thống

Xây dựng quy trình đăng ký khép kín: Phát triển đầy đủ các chức năng cho phép sinh viên đăng nhập hệ thống, tra cứu danh sách môn học theo mã hoặc tên, thực hiện đăng ký và hủy môn học trực tuyến.

Kiểm soát nghiệp vụ thông minh: Cài đặt bộ máy kiểm tra tự động nhằm đảm bảo các quy tắc bắt buộc của phòng đào tạo: giới hạn tín chỉ tối đa, ngăn chặn đăng ký trùng lịch học, kiểm soát sĩ số lớp và loại bỏ việc đăng ký trùng lặp.

Quản lý dữ liệu và nhật ký: Thiết lập cơ chế lưu trữ bền vững cho các đối tượng và ghi lại nhật ký các hoạt động đăng ký/hủy để phục vụ công tác giám sát.

1.2.2 Về kỹ thuật và kiến trúc phần mềm

Chuẩn hóa tư duy OOP: Áp dụng triệt để 4 tính chất nền tảng của lập trình hướng đối tượng là Đóng gói, Kế thừa, Đa hình và Trừu tượng trong việc thiết kế các Class và Interface.

Triển khai kiến trúc 3 tầng: Xây dựng hệ thống tách biệt rõ ràng giữa tầng Frontend (Giao diện người dùng), Backend (Xử lý nghiệp vụ với Spring Boot) và Mook Database (Lưu trữ qua file JSON), giao tiếp thống nhất qua giao thức HTTP.

Tổ chức mã nguồn khoa học: Phân chia hệ thống thành các package chức năng như model, service, repository, handler và utils, tuân thủ các nguyên tắc thiết kế như Separation of Concerns và Dependency Direction để tăng khả năng bảo trì.

1.3 Đối tượng sử dụng

Hệ thống được thiết kế và tối ưu hóa trải nghiệm để phục vụ nhóm đối tượng người dùng chính, được chuẩn hóa thông qua lớp trừu tượng User trong cấu trúc mã nguồn

Sinh viên (Student): Là nhóm người dùng trung tâm và trực tiếp thao tác trên phần mềm. Mục đích sử dụng của sinh viên là tra cứu danh sách các

môn học đang mở trong học kỳ, tìm kiếm môn học theo mã hoặc tên để nắm bắt lịch học và sĩ số. Sinh viên có quyền thực hiện đăng ký môn học mới hoặc hủy các môn đã đăng ký trước đó, đồng thời quản lý tiến độ tích lũy tín chỉ cá nhân của mình. Các thao tác này được kiểm soát nghiêm ngặt bởi bộ lọc CourseValidator để đảm bảo không vi phạm các quy tắc của phòng đào tạo.

CHƯƠNG 2: PHÂN TÍCH YÊU CẦU HỆ THỐNG

2.1 Yêu cầu chức năng

Hệ thống được thiết kế với các nhóm chức năng chính sau:

Quản lý sinh viên: Hệ thống lưu trữ các thông tin định danh như mã sinh viên, họ tên, lớp, ngành học và đặc biệt là số tín chỉ tối đa được phép đăng ký. Sinh viên có quyền truy cập để xem danh sách môn học đang mở, thực hiện đăng ký/hủy môn và kiểm soát tổng số tín chỉ đã tích lũy của bản thân.

Quản lý môn học: Mỗi học phần được định nghĩa chi tiết bao gồm mã môn, tên môn, số tín chỉ, giảng viên phụ trách, lịch học và giới hạn sĩ số (tối đa và hiện tại). Hệ thống cung cấp công cụ tìm kiếm môn học theo mã hoặc tên để hỗ trợ sinh viên trong quá trình tra cứu.

Quản lý đăng ký và hủy môn: Đây là chức năng cốt lõi. Khi sinh viên thực hiện đăng ký, hệ thống sẽ thực hiện một chuỗi kiểm tra tính hợp lệ trước khi cập nhật dữ liệu. Ngược lại, chức năng hủy môn sẽ giải phóng chỗ trong lớp học và cập nhật lại trạng thái đăng ký của sinh viên.

2.2. Yêu cầu phi chức năng

Bên cạnh các nghiệp vụ cốt lõi, hệ thống cũng đáp ứng các tiêu chuẩn cơ bản về mặt kỹ thuật phần mềm:

Khả năng bảo trì và mở rộng (Maintainability): Hệ thống được xây dựng dựa trên kiến trúc 3 tầng và tuân thủ chặt chẽ các nguyên lý OOP. Việc sử dụng các Interface (như CourseValidator) giúp dễ dàng bổ sung các quy tắc đăng ký mới trong tương lai mà không cần can thiệp vào mã nguồn cũ.

Tính lưu trữ bền vững (Persistence): Toàn bộ dữ liệu phiên làm việc được đồng bộ hóa và lưu trữ an toàn dưới dạng các tệp JSON, đảm bảo không thất thoát dữ liệu khi khởi động lại server.

Trải nghiệm người dùng (Usability): Các lỗi nghiệp vụ (như trùng lịch, vượt tín chỉ) được bọc trong các đối tượng Exception tùy chỉnh, giúp hệ thống trả về thông báo lỗi rõ ràng, tường minh cho phía giao diện.

2.3 Các quy tắc nghiệp vụ

Để đảm bảo tính đúng đắn của dữ liệu và công bằng trong đào tạo, hệ thống cài đặt các bộ lọc bắt buộc:

Quy tắc tồn tại: Không cho phép đăng ký môn học không có trong hệ thống.

Quy tắc sĩ số: Không cho phép đăng ký khi lớp học đã đủ số lượng sinh viên tối đa.

Quy tắc trùng lặp: Sinh viên không được phép đăng ký cùng một môn học nhiều lần.

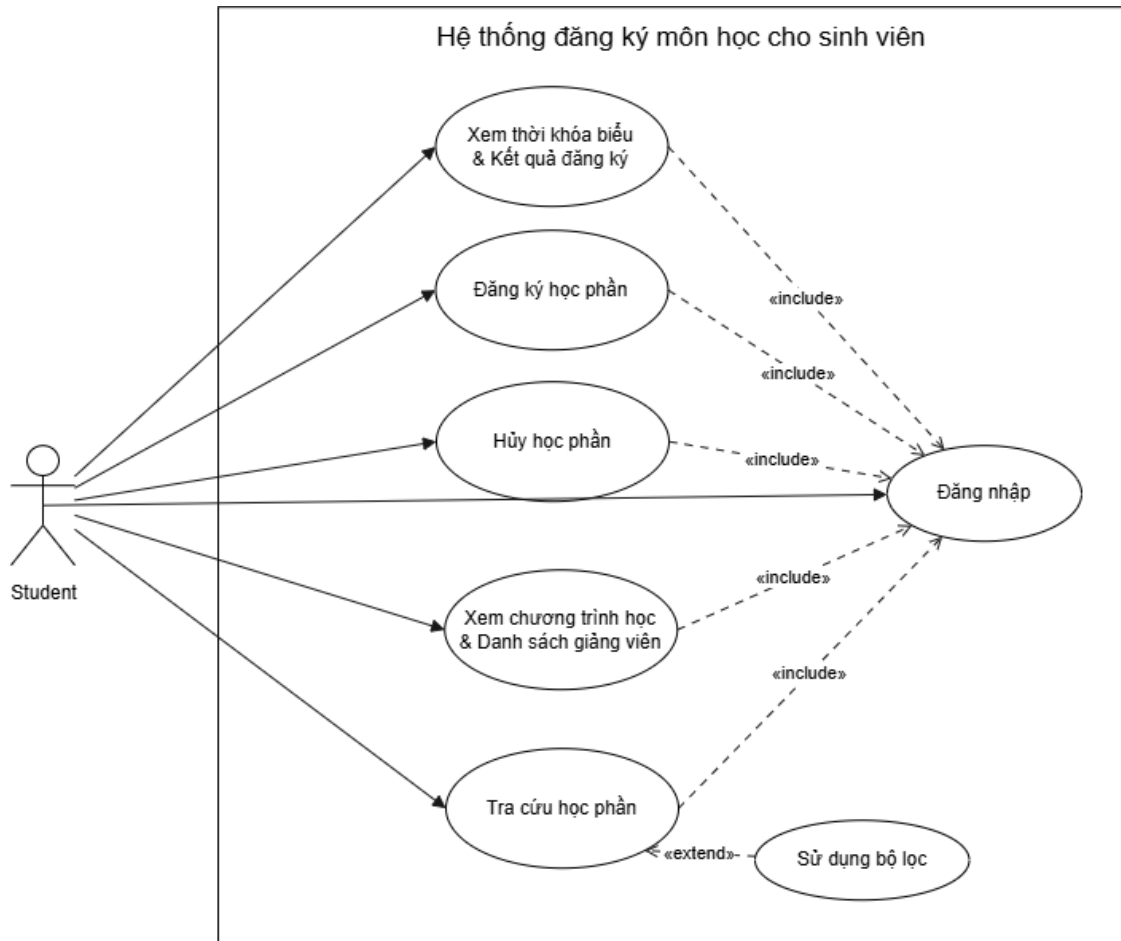
Quy tắc tín chỉ: Tổng số tín chỉ sau khi đăng ký mới không được vượt giới hạn cho phép của từng sinh viên.

Quy tắc lịch học: Ngăn chặn việc đăng ký các môn học có lịch học trùng hoặc chồng chéo thời gian với các môn học đã đăng ký thành công.

Quy tắc hủy môn: Chỉ cho phép hủy những môn học mà sinh viên đã đăng ký hợp lệ trước đó.

2.4 Biểu đồ Use Case

Biểu đồ Use Case tổng quát dưới đây mô phỏng lại toàn bộ các tương tác chính giữa các nhóm người dùng và Hệ thống đăng ký môn học. Biểu đồ thể hiện sự phân quyền và giới hạn chức năng của từng đối tượng khi tham gia vào hệ thống.



Hình 2.1 Biểu đồ Use Case tổng quát của hệ thống

Trong phạm vi của hệ thống Đăng ký môn học, người dùng tương tác trực tiếp với giao diện Frontend được xác định là một tác nhân duy nhất:

Student (Sinh viên): Là tác nhân chính của hệ thống, đại diện cho đối tượng người dùng có nhu cầu thực hiện các nghiệp vụ liên quan đến lộ trình học tập, đăng ký tín chỉ và theo dõi thời khóa biểu cá nhân.

Các chức năng dành cho tác nhân Sinh viên bao gồm:

- **Đăng nhập:** Là chức năng tiên quyết và bắt buộc. Sinh viên phải xác thực danh tính bằng mã sinh viên (Username) và mật khẩu (Password) hợp lệ trước khi được cấp quyền sử dụng hệ thống.
- **Xem chương trình học & Danh sách giảng viên:** Cho phép Sinh viên tra cứu thông tin tổng quan về khung chương trình đào tạo của ngành học và thông tin chuyên môn của các giảng viên phụ trách.
- **Tra cứu học phần:** Cho phép Sinh viên xem danh sách chi tiết các lớp học phần đang được mở đăng ký trong học kỳ. Đặc biệt, ca sử dụng này được mở rộng bởi chức năng Sử dụng bộ lọc, hỗ trợ Sinh viên dễ dàng tìm kiếm lớp học theo tiêu chí cụ thể như tên giảng viên hoặc thứ học trong tuần.
- **Đăng ký học phần:** Là chức năng cốt lõi (xương sống) của hệ thống. Khi Sinh viên thao tác chọn lớp, quá trình ghi nhận sẽ được hệ thống kiểm soát ngầm thông qua các bộ lọc điều kiện nghiệp vụ khắt khe (kiểm tra lớp còn chỗ, không trùng lịch học, không vượt giới hạn tín chỉ tối đa) nhằm đảm bảo tính toàn vẹn và hợp lệ của dữ liệu.
- **Hủy học phần:** Cho phép Sinh viên rút khỏi một lớp học phần. Hệ thống sẽ kiểm tra xác nhận sinh viên đã đăng ký thành công môn học này trước đó, sau đó tiến hành hủy bản ghi và hoàn trả lại chỗ trống (slot) cho lớp học.
- **Xem thời khóa biểu & Kết quả đăng ký:** Chức năng tổng hợp đầu ra, cho phép Sinh viên theo dõi chi tiết lịch học (thứ, ca, phòng học) của các học phần đã đăng ký, đồng thời kiểm soát tổng số tín chỉ hiện tại để lên kế hoạch học tập tối ưu nhất.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

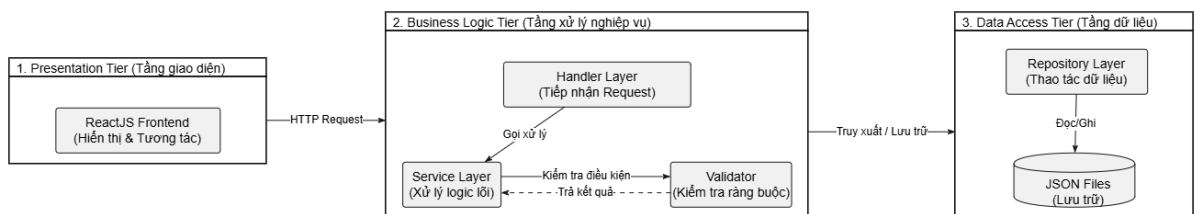
3.1 Kiến trúc phần mềm

Để đảm bảo tính mở rộng, dễ bảo trì và tuân thủ các tiêu chuẩn thiết kế phần mềm hiện đại, dự án áp dụng kiến trúc 3 tầng. Việc phân tách rõ ràng trách nhiệm giữa các tầng giúp hệ thống giảm thiểu sự phụ thuộc lẫn nhau, hỗ trợ việc làm nhóm hiệu quả khi từng thành viên có thể tập trung vào module riêng biệt. Kiến trúc cụ thể được chia thành:

Presentation(Tầng giao diện): Được xây dựng bằng thư viện ReactJS, chịu trách nhiệm hiển thị thông tin và tiếp nhận tương tác từ người dùng. Tầng này giao tiếp với tầng xử lý thông qua các HTTP Request.

Business Logic Tier(Tầng xử lý nghiệp vụ): Đây là “não bộ” của hệ thống, được phát triển trên nền tảng Java OOP(Spring Boot). Tầng này tiếp nhận yêu cầu từ Frontend (thông qua các lớp Handler), xử lý các logic nghiệp vụ phức tạp, kiểm tra các điều kiện ràng buộc (thông qua các lớp Service, Validator) trước khi quyết định thay đổi dữ liệu.

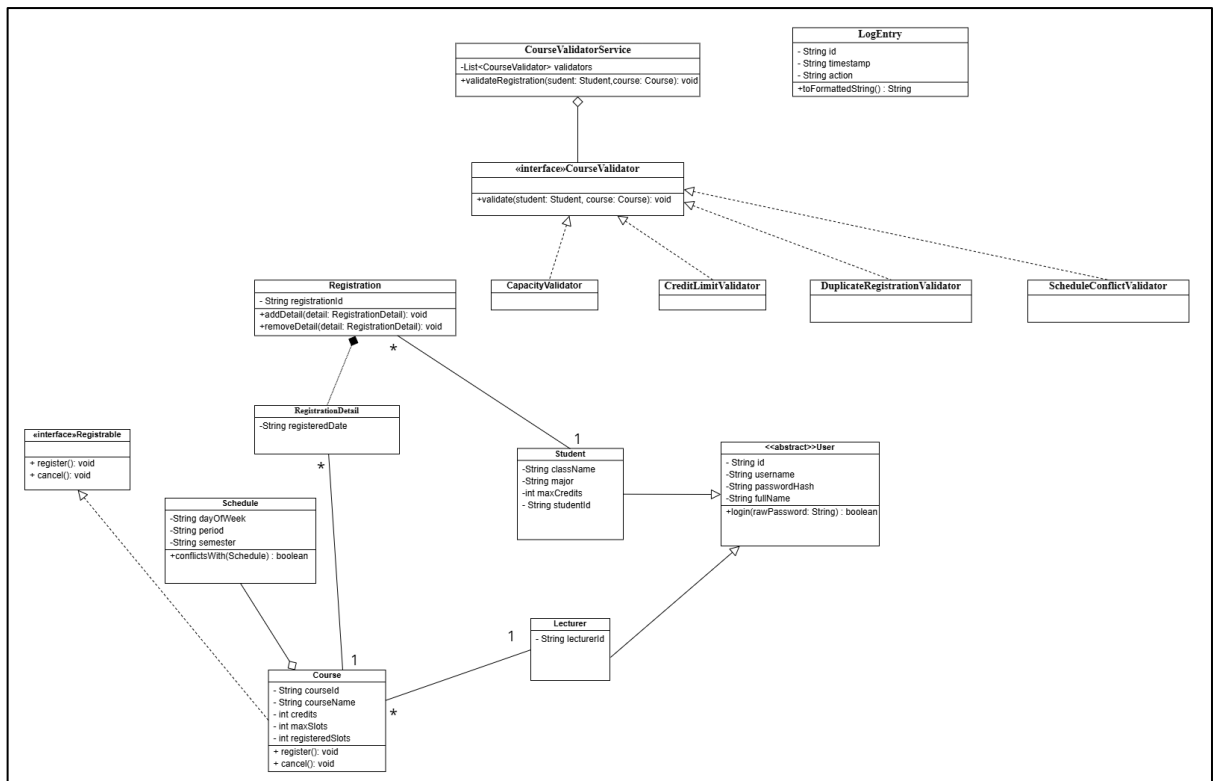
Data Access Tier(Tầng dữ liệu): Xử lý việc truy xuất và lưu trữ dữ liệu. Thay vì sử dụng cơ sở dữ liệu quan hệ công kênh, hệ thống thao tác với các tệp JSON thông qua các lớp Repository.



Hình 3.1 Sơ đồ kiến trúc 3 tầng của hệ thống

3.2 Class Diagram

Biểu đồ lớp (Class Diagram) là bản thiết kế tĩnh của toàn bộ hệ thống, minh họa cách các thực thể trong không gian bài toán được mô hình hoá thành các đối tượng phần mềm, đồng thời thể hiện việc áp dụng triệt để 4 nguyên lý của Lập trình hướng đối tượng



Hình 3.2 Biểu đồ lớp mô tả cấu trúc tĩnh và quan hệ các đối tượng

Áp dụng tư duy OOP trong thiết kế lớp:

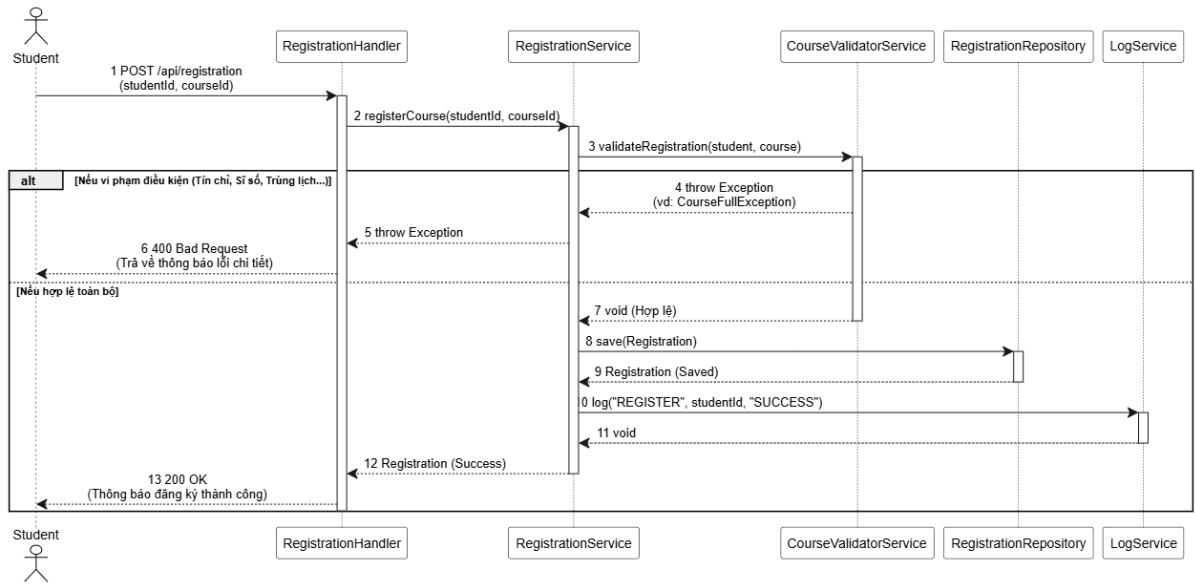
Tính Trừu tượng và kế thừa: Xây dựng lớp User chứa các thuộc tính chung định danh. Từ đó, các lớp Student và Lecturer kế thừa User để tái sử dụng mã nguồn và phát triển thêm các thuộc tính riêng.

Tính Đóng gói: Toàn bộ thuộc tính của các lớp đều được đặt ở mức truy cập private và thao tác thông qua hệ thống getter/setter hoặc các phương thức nghiệp vụ, đảm bảo dữ liệu không bị sửa đổi tùy tiện.

Tính Đa hình: Hệ thống sử dụng các interface cho phép gọi các hành vi một cách thống nhất mà không cần phụ thuộc vào chi tiết cài đặt của từng lớp con.

3.3 Sequence Diagram

Để làm rõ cách hoạt động của hệ thống, nhóm lựa chọn minh họa luồng nghiệp vụ phức tạp và quan trọng nhất: Đăng ký môn học. Biểu đồ tuần tự dưới đây thể hiện sự tương tác theo thời gian thực giữa các đối tượng ở tầng logic.



Hình 3.3 Biểu đồ tuần tự biểu diễn luồng nghiệp vụ đăng ký học phần

Mô tả luồng xử lý:

1. Sinh viên gửi yêu cầu POST đăng ký học phần kèm studentID và courseId.
2. RegistrationHandler tiếp nhận yêu cầu và gọi đến RegistrationService.
3. RegistrationService không tự xử lý ngay mà gọi đến CourseValidator để chạy một chuỗi các bước kiểm tra:
 - CreditLimitValidator: Kiểm tra tín chỉ vượt mức.
 - CapacityValidator: Kiểm tra sĩ số lớp học.
 - ScheduleConflictValidator: Kiểm tra trùng lịch.
 - DuplicateRegistrationValidator: Kiểm tra đăng ký lặp.
4. Nếu bất kỳValidator nào báo lỗi, hệ thống ném ngoại lệ và trả về thông báo từ chối sinh viên.
5. Nếu hợp lệ toàn bộ, RegistrationService gọi RegistrationRepository để ghi nhận phiếu đăng ký, đồng thời gọi LogService để ghi lại nhật ký hành động trên hệ thống.

3.4 Mock Database

Do tính chất mô phỏng của đề tài và để tối ưu hoá việc di chuyển mã nguồn, dự án sử dụng cơ chế Mock Database dựa trên định dạng tệp JSON. Cơ chế này mang lại khả năng lưu trữ dữ liệu bền vững tương tự như Database vật lý nhưng nhẹ và dễ dàng quản lý hệ thống thông qua thư viện Gson của Java.

Cấu trúc lưu trữ được chia thành các Collection độc lập, bao gồm:

- Students.json, lecturer.json: Lưu trữ thông tin tài khoản người dùng.
- Courses.json: Chứa thông tin học phần, sĩ số hiện tại và lịch học
- Registrations.json: Lưu trữ chi tiết các phiếu đăng ký môn học của từng sinh viên.
- Logs.json: Ghi nhận lịch sử thao tác toàn hệ thống phục vụ truy vết.

Việc đọc/ghi vào các tệp này được cách ly hoàn toàn ở tầng Repository, giúp cho việc nâng cấp lên một hệ quản trị cơ sở dữ liệu thật (như MySQL/PostgreSQL) trong tương lai có thể thực hiện dễ dàng mà không làm ảnh hưởng đến mã nguồn tầng Service.

CHƯƠNG 4: TỔ CHỨC MÃ NGUỒN VÀ CÔNG NGHỆ

4.1 Cấu trúc Package

Mã nguồn của Backend được phân bổ vào các package độc lập, tạo thành một hệ sinh thái khép kín nhưng giao tiếp mượt mà qua các Interface và Method công khai:

- **Package model/ (Tầng thực thể):** Đây là phần cốt lõi của hệ thống, nơi ánh xạ các đối tượng thực tế vào không gian phần mềm. Tính đóng gói được áp dụng triệt để với toàn bộ trạng thái của đối tượng (như maxCredits của sinh viên hay registeredSlots của môn học) đều được khai báo private và chỉ cho phép thay đổi thông qua các phương thức nghiệp vụ hoặc getter/setter được kiểm soát. Đặc biệt, tính kế thừa được thể hiện qua việc lớp Student và Lecturer cùng kế thừa lớp trừu tượng User, giúp loại bỏ sự trùng lặp mã nguồn trong việc quản lý thông tin định danh và tài khoản đăng nhập.
- **Package repository/ (Tầng truy xuất dữ liệu):** Chịu trách nhiệm duy nhất là tương tác với hệ thống lưu trữ. Việc tách biệt UserRepository, CourseRepository và RegistrationRepository giúp cô lập hoàn toàn logic đọc/ghi tệp JSON khỏi phần xử lý nghiệp vụ. Thiết kế này tuân thủ nguyên lý Dependency Inversion, cho phép hệ thống dễ dàng thay thế Mock Database bằng một hệ quản trị cơ sở dữ liệu thực (như SQL) trong tương lai mà không làm ảnh hưởng đến các module khác.
- **Package service/ (Tầng xử lý nghiệp vụ):** Đây là nơi chứa đựng “trí tuệ” của hệ thống, xử lý các logic phức tạp như tính toán tín chỉ(SummaryService), xác thực tài khoản(AuthService) và xử lý huỷ đăng ký(CancellationService). Tại đây, tính đa hình được thể hiện thông qua CourseValidatorService. Bằng cách khai báo lớp interface CourseValidator, hệ thống có thể chạy tuần tự hàng loạt các lớp kiểm tra độc lập (như CreditLimitValidator, CapacityValidator). Nếu phòng đào tạo ban hành quy chế mới, lập trình viên chỉ cần tạo thêm một Validator mới implement interface này mà không cần sửa đổi mã nguồn cũ.
- **Package handler/ (Tầng điều hướng API):** Tầng này đóng vai trò như các trạm kiểm soát, tiếp nhận HTTP Request từ giao diện người

dùng, chuyển tiếp dữ liệu vào tầng service xử lý và trả về HTTP Response định dạng JSON. Lớp này tuyệt đối không chứa logic nghiệp vụ, đảm bảo tính mạch lạc của luồng dữ liệu.

- Package exception/ (Quản lý ngoại lệ hướng đối tượng): Thay vì sử dụng các lỗi hệ thống chung chung, dự án xây dựng một cây kế thừa ngoại lệ riêng biệt như CourseFullException, CreditLimitExceededException, DuplicateRegistrationException và ScheduleConflictException. Việc bọc lỗi vào các đối tượng Exception cụ thể giúp tầng Handler dễ dàng bắt và trả về thông báo lỗi chính xác cho Frontend.
- Package utils/ (Tiện ích dùng chung): Chứa các lớp công cụ độc lập như FileUtils (hỗ trợ đọc/ghi bằng Gson), IdGenerator(sinh mã tự tăng) và PasswordUtils(hỗ trợ mã hoá bảo mật).
- Package data/: Thư mục vật lý chứa các tệp cơ sở dữ liệu giả lập như student.json, courses.json, registrations.json và logs.json.

4.2 Các công nghệ và công cụ sử dụng

Để hiện thực hoá kiến trúc trên, dự án đã áp dụng chuỗi công nghệ và tiêu chuẩn phát triển phần mềm hiện đại, đáp ứng yêu cầu của một ứng dụng Web chuyên nghiệp.

4.2.1 Công nghệ Backend

Ngôn ngữ Java (phiên bản 17+): Ngôn ngữ nền tảng của dự án, cung cấp bộ công cụ mạnh mẽ để cài đặt thiết kế hướng đối tượng chặt chẽ từ trừu tượng đến đa hình.

Spring Boot (Spring Web): Framework mạnh mẽ được sử dụng để khởi tạo server nhanh chóng, cấu hình CORS và xây dựng hệ thống RESTful API chuẩn mực nhằm giao tiếp với Frontend.

Maven: Công cụ quản lý dự án và đóng gói phần mềm, giúp tự động hoá việc tải và quản lý các thư viện phụ thuộc.

Thư viện Gson: Công cụ đắc lực do Google phát triển, đảm nhiệm việc chuyển đổi qua lại giữa các đối tượng Java và cấu trúc dữ liệu JSON để lưu trữ xuống tệp.

4.2.2 Công nghệ Frontend

Sử dụng ReactJS & Vite cho giao diện hệ thống (LoginPage.jsx, CoursePage.jsx, MyRegistrationsPage.jsx, LecturerPage.jsx) được xây dựng dựa trên ReactJS, ứng dụng kiến trúc Component để tái sử dụng giao diện. Kết hợp cùng công cụ Vite giúp tăng tốc độ build dự án, mang lại trải nghiệm tương tác mượt mà và trực quan cho người sử dụng.

4.2.3 Quản lý dự án và mã nguồn

Git & GitHub: Công cụ quản trị phiên bản phân tán bắt buộc phải có trong quy trình làm việc nhóm. GitHub đóng vai trò là kho lưu trữ tập trung, cho phép 6 thành viên trong nhóm làm việc song song trên các nhánh khác nhau, giải quyết xung đột mã nguồn và theo dõi tiến độ công việc một cách minh bạch và trực quan.

CHƯƠNG 5: CÀI ĐẶT CHI TIẾT VÀ ÁP DỤNG TƯ DUY OOP

5.1 Áp dụng các nguyên lý OOP

5.1.1 Tính đóng gói

Tính đóng gói (Encapsulation) được áp dụng triệt để tại toàn bộ thực thể trong hệ thống nhằm bảo vệ toàn vẹn dữ liệu. Toàn bộ các trạng thái của đối tượng như maxCredits của sinh viên, registeredSlots của môn học hay passwordHash của người với từ khóa private. Việc truy xuất hay thay đổi các giá trị này từ bên ngoài lớp không được thực hiện trực tiếp mà bắt buộc phải thông qua hệ thống các phương thức getter/setter hoặc các phương thức nghiệp vụ cụ thể. Điều này đảm bảo dữ liệu không bị sửa đổi tùy tiện, sai lệch logic hệ thống.

5.1.2 Tính kế thừa

Nguyên lý kế thừa (Inheritance) được tận dụng để tối ưu hóa mã nguồn và quản lý cấu trúc phân cấp người dùng. Hệ thống định nghĩa một lớp trừu tượng User chứa các thuộc tính định danh chung như id, username, passwordHash và fullName. Từ đó, các lớp Student và Lecturer được thiết kế kế thừa trực tiếp từ lớp User, giúp tái sử dụng toàn bộ cấu trúc mã nguồn chung, đồng thời mở rộng thêm các thuộc tính chuyên biệt như maxCredits (với sinh viên) hay lecturerId (với giảng viên). Ngoài ra, tính kế thừa còn được áp dụng trong việc xây dựng cây quản lý ngoại lệ, trong đó các ngoại lệ tùy chỉnh đều kế thừa từ lớp RuntimeException.

5.1.3 Tính đa hình và trừu tượng

Tính đa hình (Polymorphism) và tính trừu tượng (Abstraction) đóng vai trò cốt lõi trong việc tạo ra một kiến trúc mềm dẻo, dễ mở rộng. Tính trừu tượng được thể hiện thông qua việc sử dụng lớp trừu tượng User và interface CourseValidator (chỉ định nghĩa hàm validate() mà không cài đặt chi tiết). Tính đa hình tỏa sáng tại module kiểm tra điều kiện đăng ký, nơi hệ thống có thể gọi phương thức validate() trên một danh sách các bộ lọc (List<CourseValidator>) một cách thống nhất, mà không cần bận tâm đối tượng thực sự đang thực thi tại thời điểm chạy (Runtime) là CreditLimitValidator hay CapacityValidator.

5.2 Cài đặt các Module chức năng

5.2.1 Module Xác thực và thông tin sinh viên

Module Xác thực và thông tin sinh viên chịu trách nhiệm quản lý chức năng đăng nhập của người dùng và cung cấp các thông tin cơ bản của sinh viên trong hệ thống. Đây là một trong những module nền tảng, đóng vai trò xác thực danh tính người dùng trước khi cho phép truy cập vào các chức năng nghiệp vụ khác của hệ thống.

Trong tầng Model, lớp trừu tượng User được hoàn thiện để quản lý các thông tin chung của người dùng như mã định danh, tên đăng nhập, mật khẩu đã mã hóa và họ tên. Phương thức login() được xây dựng nhằm kiểm tra tính hợp lệ của mật khẩu thông qua lớp tiện ích PasswordUtils. Lớp Student kế thừa từ User, bổ sung các thuộc tính riêng gồm mã sinh viên, lớp, ngành học và số tín chỉ tối đa được phép đăng ký. Việc kế thừa giúp tái sử dụng các thuộc tính và hành vi chung của người dùng, đồng thời mở rộng thêm các đặc trưng riêng của sinh viên.

Trong tầng Repository, lớp UserRepository đảm nhiệm việc truy xuất dữ liệu sinh viên từ tệp JSON. Module cung cấp các phương thức findByUsername(), findById() và save() để tìm kiếm hoặc cập nhật thông tin sinh viên. Việc tách riêng tầng Repository giúp toàn bộ thao tác đọc và ghi dữ liệu được quản lý tập trung, giảm sự phụ thuộc giữa tầng nghiệp vụ và tầng lưu trữ.

Trong tầng Service, lớp AuthService thực hiện chức năng xác thực người dùng. Khi nhận yêu cầu đăng nhập, hệ thống tìm kiếm tài khoản theo tên đăng nhập, sau đó sử dụng PasswordUtils để so sánh mật khẩu người dùng nhập với giá trị đã được băm (hash) trong hệ thống. Chỉ khi hai giá trị trùng khớp thì quá trình đăng nhập mới được chấp nhận.

Bên cạnh đó, lớp StudentService cung cấp các nghiệp vụ liên quan đến sinh viên như lấy thông tin sinh viên theo mã định danh và tính tổng số tín chỉ đã đăng ký. Việc tính tổng tín chỉ được thực hiện bằng cách duyệt danh sách đăng ký môn học của sinh viên và cộng tổng số tín chỉ của các học phần đã đăng ký thành công. Kết quả này có thể được các module nghiệp vụ khác sử dụng để kiểm tra giới hạn số tín chỉ khi sinh viên đăng ký thêm môn học.

Ở tầng Handler, lớp AuthHandler xây dựng REST API POST /api/login để tiếp nhận yêu cầu đăng nhập từ phía giao diện người dùng. Sau khi xác thực thành công, hệ thống trả về thông tin người dùng tương ứng; nếu thông tin đăng nhập không hợp lệ, hệ thống trả về phản hồi lỗi phù hợp. Đồng thời, lớp StudentHandler cung cấp các API GET /api/students/{id} và GET /api/students/{id}/credits nhằm truy xuất thông tin sinh viên và tổng số tín chỉ đã đăng ký.

Thông qua module này, các nguyên lý của lập trình hướng đối tượng được áp dụng rõ ràng. Tính đóng gói được thể hiện bằng việc các thuộc tính của lớp được khai báo ở mức private và truy cập thông qua các phương thức getter và setter. Tính kế thừa được sử dụng khi lớp Student kế thừa từ lớp trừu tượng User, giúp tái sử dụng mã nguồn và giảm trùng lặp. Tính trừu tượng được áp dụng thông qua lớp User, đóng vai trò mô tả các đặc điểm chung của người dùng trong hệ thống. Bên cạnh đó, việc phân chia hệ thống thành các tầng Repository, Service và Handler giúp tách biệt rõ ràng giữa lưu trữ dữ liệu, xử lý nghiệp vụ và cung cấp dịch vụ REST, từ đó nâng cao khả năng bảo trì và mở rộng hệ thống trong tương lai.

5.2.2 Module Quản lý môn học và lịch học

5.2.2.1 Mô tả chức năng

Module này đảm nhận nhiệm vụ quản lý toàn bộ cơ sở dữ liệu về danh mục học phần, thông tin chi tiết lớp học phần (như mã môn, tên môn, số tín chỉ, giảng viên, sĩ số) và xây dựng thời khóa biểu cá nhân cho sinh viên. Chức năng này cho phép sinh viên dễ dàng tra cứu danh sách các môn học được mở trong học kỳ, lọc theo giảng viên hoặc thời gian, đồng thời trực quan hóa lịch học hàng tuần để theo dõi tiến độ học tập một cách khoa học.

5.2.2.2 Các class/method chính đã viết

Lớp thực thể Course và CourseClass: Định nghĩa cấu trúc dữ liệu cho môn học và lớp học phần, lưu trữ các thông tin cốt lõi như mã học phần, tên học phần, số tín chỉ (credits), số lượng đăng ký tối đa và thời gian học (Schedule).

CourseRepository / CourseManager: Chịu trách nhiệm truy xuất, tìm kiếm và lọc danh sách các lớp học phân từ nguồn dữ liệu (hoặc cơ sở dữ liệu giả lập).

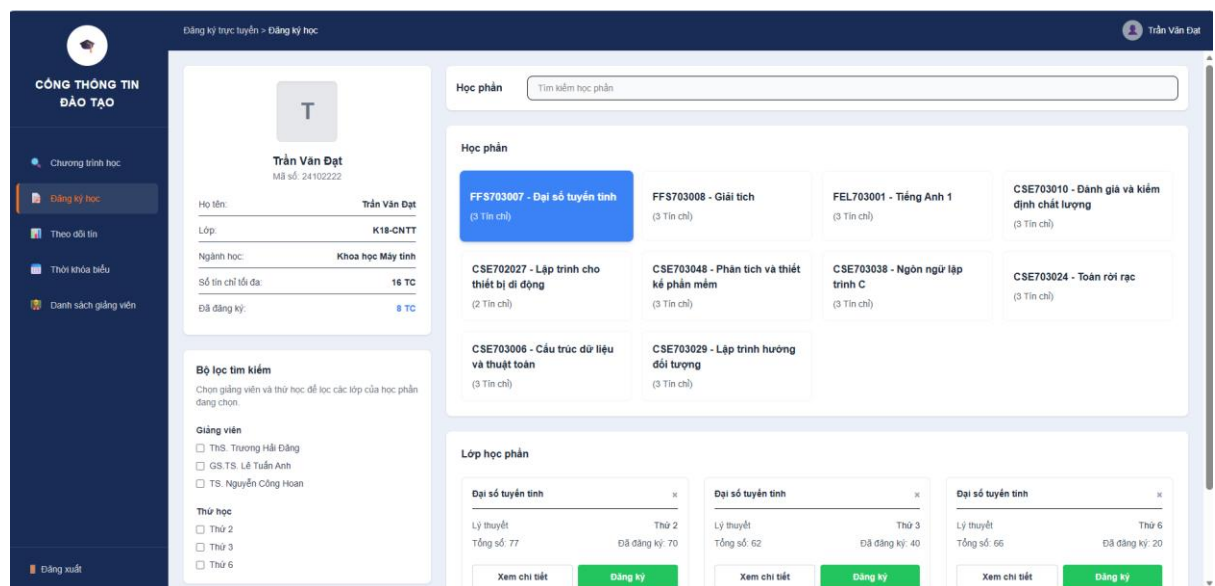
ScheduleService: Xử lý logic nghiệp vụ liên quan đến việc tổng hợp và trả về thời khóa biểu cá nhân dựa trên danh sách các học phần mà sinh viên đã đăng ký thành công.

5.2.2.3 Nguyên lý OOP áp dụng trong module này

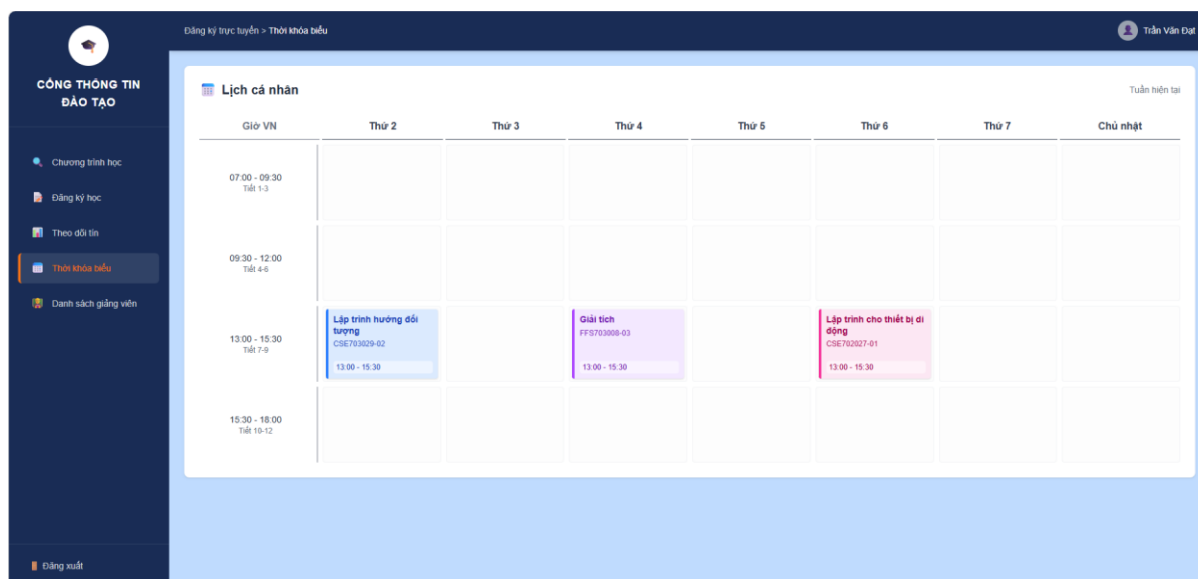
Tính đóng gói (Encapsulation): Các thuộc tính của lớp Course và CourseClass được bảo vệ ở chế độ private, việc truy xuất và chỉnh sửa dữ liệu bắt buộc phải thông qua các phương thức getter và setter chuẩn.

Tính trừu tượng (Abstraction): Tách biệt rõ ràng giữa tầng hiển thị giao diện, tầng xử lý logic nghiệp vụ (Service) và tầng dữ liệu (Repository), giúp mã nguồn dễ mở rộng và bảo trì.

5.2.2.4 Ảnh minh họa



Hình 5.1: Giao diện tra cứu danh sách lớp học phần và thông tin chi tiết môn học.



Hình 5.2: Giao diện hiển thị thời khóa biểu cá nhân của sinh viên theo tuần.

5.2.2.6 Kết quả test

Đã viết các ca kiểm thử đơn vị (Unit Test) cho các chức năng tra cứu môn học, lọc lớp học phân và truy xuất thời khóa biểu:

STT	Tên test case	Input	Kết quả mong đợi
1	testGetCourseList_ThanhCong	Yêu cầu tải danh sách toàn bộ học phần hiện có.	Trả về danh sách Course đầy đủ thông tin, không bị rỗng (not null).
2	testFilterCourseByInstructor	Tên giảng viên cần tìm kiếm	Lọc chính xác và trả về danh sách các lớp học phân do đúng giảng viên đó phụ trách.
	testGetSchedule_ThanhCong	Mã sinh viên đã đăng ký học phần	Trả về danh sách thời khóa biểu tương ứng với các môn đã đăng ký thành công.

Bảng 5.1 Bảng kiểm thử module quản lý môn học và lịch học

Kết quả chạy mvn test: Các ca kiểm thử cho module quản lý môn học và lịch học đều phản hồi chính xác, đảm bảo dữ liệu hiển thị đồng bộ và đạt trạng thái **BUILD SUCCESS**.

5.2.3 Module Kiểm tra điều kiện đăng ký

5.2.3.1 Mô tả chức năng

Module đóng vai trò là "màng lọc tự động" của hệ thống trước khi chấp nhận bất kỳ phiếu đăng ký môn học nào của sinh viên. Nhiệm vụ của module là kiểm tra tính hợp lệ của yêu cầu đăng ký dựa trên hàng loạt các quy tắc ràng buộc khắt khe do phòng đào tạo quy định (như giới hạn tổng số tín chỉ, kiểm tra sĩ số lớp học, ngăn chặn trùng lặp môn học và xung đột thời khóa biểu). Nếu phát sinh bất kỳ vi phạm nào, hệ thống sẽ lập tức chặn lại và ném ra ngoại lệ tương ứng.

5.2.3.2 Các class/method chính:

Interface CourseValidator: Định nghĩa bản thiết kế chung với phương thức validate(Student student, Course course).

Các lớp Validator cụ thể: Cài đặt trực tiếp interface trên để xử lý từng quy tắc độc lập bao gồm: CreditLimitValidator (chặn vượt quá tổng tín chỉ maxCredits), CapacityValidator (chặn khi sĩ số registeredSlots đã đạt maxSlots), ScheduleConflictValidator (chặn trùng lặp thời gian học), và DuplicateRegistrationValidator (ngăn sinh viên đăng ký hai lần một môn).

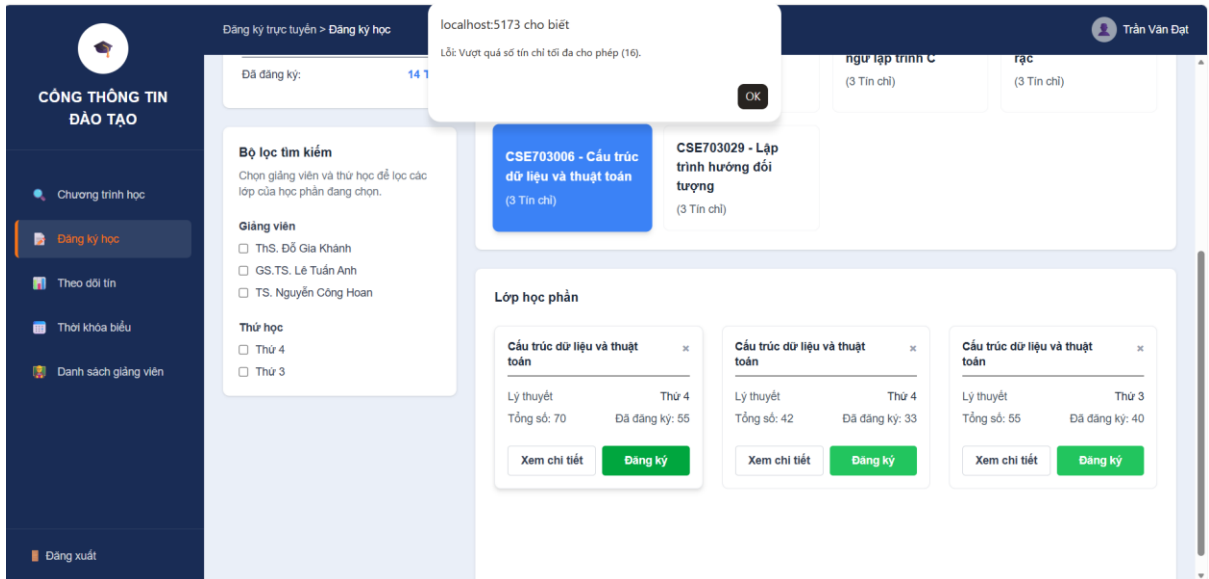
CourseValidatorService: Đóng vai trò là lớp quản lý trung tâm. Lớp này chứa danh sách các validator (List<CourseValidator>) và sở hữu phương thức validateAll(Student, Course). Phương thức này sẽ lặp tuần tự qua các bộ lọc, nếu phát hiện vi phạm sẽ lập tức ném ra ngoại lệ (throw Exception) để báo lỗi ngay cho tầng Handler.

5.2.3.3 Nguyên lý OOP áp dụng trong module này

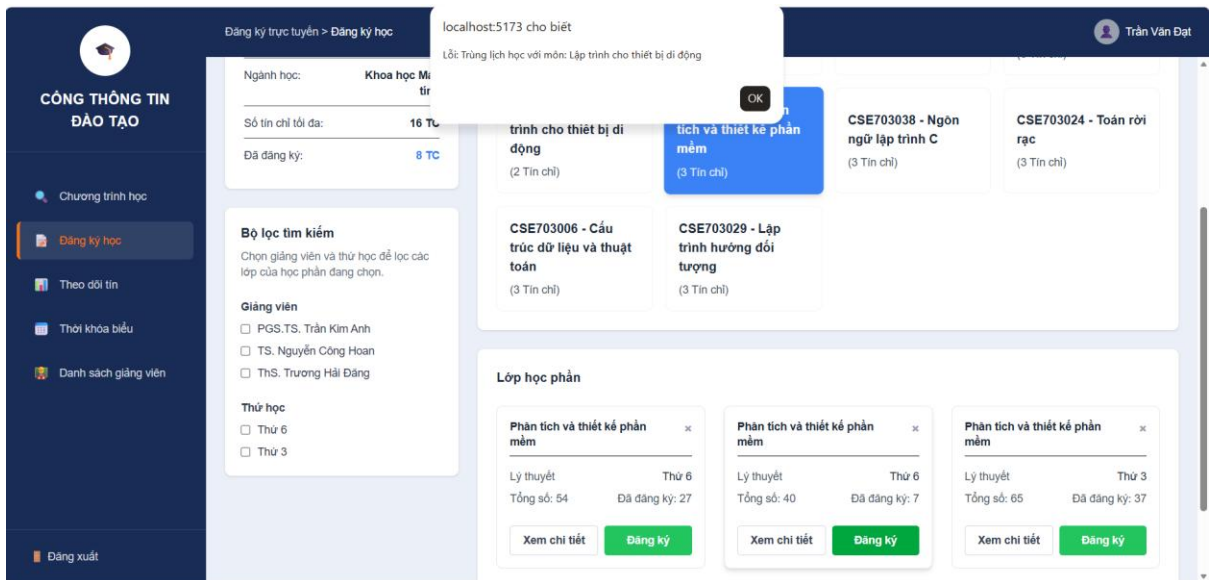
Tính đa hình (Polymorphism) & Trừu tượng (Abstraction): Thông qua interface CourseValidator, CourseValidatorService có thể gọi phương thức validate() một cách thống nhất trên mọi bộ lọc mà không cần quan tâm chi tiết bên trong của từng lớp con.

Nguyên lý đóng/mở (Open/Closed Principle - OCP): Thiết kế cho phép dễ dàng mở rộng thêm các quy tắc kiểm tra mới trong tương lai bằng cách viết thêm một lớp implement interface CourseValidator mà hoàn toàn không phải sửa đổi mã nguồn cũ.

5.2.3.4 Ảnh minh họa



Hình 5.2 Giao diện thông báo lỗi khi sinh viên đăng ký vượt quá giới hạn số tín chỉ cho phép.



Hình 5.3 Giao diện thông báo lỗi khi hệ thống phát hiện xung đột lịch học của học phần.

5.2.3.5 Kết quả test

Đã xây dựng và thực thi các ca kiểm thử đơn vị (Unit Test) cho từng bộ lọc điều kiện đăng ký:

ST T	Tên test case	Input	Kết quả mong đợi
1	testCreditLimitValidator_VuotTinChi	Sinh viên đăng ký vượt quá maxCredits cho phép	Ném ngoại lệ CreditLimitExceededException, từ chối đăng ký.
2	testCapacityValidator_LopDay	Đăng ký vào lớp học phần đã đủ maxSlots	Ném ngoại lệ CourseFullException, từ chối đăng ký.
3	testScheduleConflictValidator_TrungLich	Đăng ký môn có thời gian trùng với lịch đã học	Ném ngoại lệ ScheduleConflictException, từ chối đăng ký.

Bảng 5.2 Bảng kiểm thử module kiểm tra điều kiện đăng ký

Kết quả chạy mvn test: Các ca kiểm thử của bộ lọc đều trả về ngoại lệ chính xác theo đúng kịch bản nghiệp vụ và đạt trạng thái **BUILD SUCCESS**.

5.2.4 Module Hủy đăng ký và tổng hợp kết quả

5.2.4.1 Mô tả chức năng

Module này đảm nhiệm hai nghiệp vụ chính trong hệ thống đăng ký môn học:

- Hủy đăng ký môn học: cho phép sinh viên hủy một môn học đã đăng ký trước đó. Khi hủy, hệ thống xác nhận sinh viên đã đăng ký môn này chưa, sau đó xóa môn học khỏi phiếu đăng ký và cập nhật lại số chỗ trống của lớp học phần.
- Tổng hợp kết quả đăng ký: cho phép tra cứu danh sách các môn học một sinh viên đã đăng ký, cùng tổng số tín chỉ hiện tại. Chức năng này còn được module Kiểm tra điều kiện đăng ký sử dụng để kiểm tra giới hạn tín chỉ tối đa khi đăng ký môn mới.

Hai chức năng này phục vụ tình huống: sinh viên muốn xem lại các môn đã đăng ký trong học kỳ, hoặc muốn hủy bớt một môn không còn phù hợp.

5.2.4.2 Các class/method chính đã viết

a) RegistrationRepository

- `findByStudentId(String studentId)`: tìm phiếu đăng ký của một sinh viên cụ thể trong dữ liệu JSON.
- `save(Registration registration)`: lưu lại phiếu đăng ký, cập nhật nếu đã tồn tại, thêm mới nếu chưa có.

b) CancellationService

- `cancelRegistration(String studentId, String courseId)`: tìm phiếu đăng ký của sinh viên, kiểm tra môn học cần hủy có tồn tại trong phiếu không. Nếu không hợp lệ, ném ngoại lệ `RegistrationNotFoundException`. Nếu hợp lệ, gọi `course.cancel()` để cập nhật số chỗ trống, xóa chi tiết đăng ký và lưu lại.

c) SummaryService

- `getTotalCredits(String studentId)`: tính tổng số tín chỉ các môn sinh viên đã đăng ký, được `RegistrationService` sử dụng để phối hợp với `CreditLimitValidator`.

- `getRegisteredCourses(String studentId)`: trả về danh sách chi tiết các môn học đã đăng ký.

d) RegistrationHandler

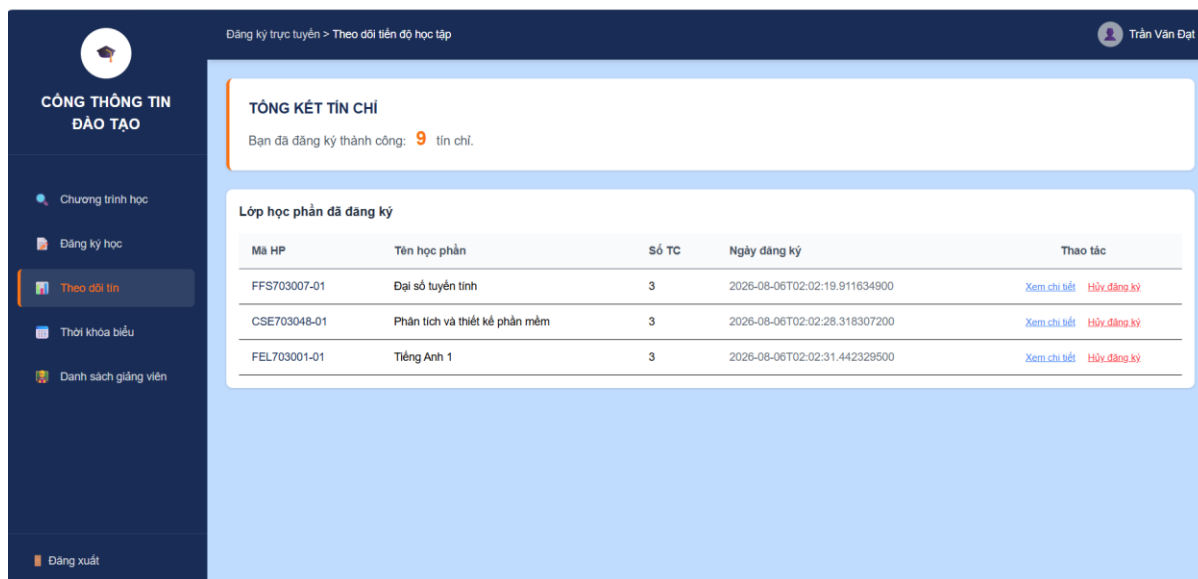
- `DELETE /api/registrations/{studentId}?courseId=...`: gọi `CancellationService` để hủy môn học.
- `GET /api/registrations/{studentId}/summary`: gọi `SummaryService` để trả về tổng tín chỉ và danh sách môn đã đăng ký.

5.2.4.3 Nguyên lý OOP áp dụng trong module này

- Encapsulation (Đóng gói): các field trong `Registration`, `RegistrationDetail`, `Course` đều ở dạng `private`, chỉ truy cập qua `getter/setter`.
- Polymorphism (Đa hình): `Course` cài đặt interface `Registrable`, cho phép gọi `cancel()` thống nhất mà không cần biết chi tiết cài đặt bên trong.
- Separation of Concerns: mỗi lớp đảm nhiệm một vai trò rõ ràng — `Repository` lo dữ liệu, `Service` lo nghiệp vụ, `Handler` lo API — giúp code dễ bảo trì, mở rộng.
- Exception Handling hướng đối tượng: sử dụng `RegistrationNotFoundException` (kế thừa `RuntimeException`) để xử lý lỗi nghiệp vụ rõ ràng, tường minh.

5.2.4.4 Ảnh minh họa

Giao diện minh họa chức năng tổng hợp kết quả đăng ký của sinh viên kết hợp với danh sách chi tiết các lớp học phần đã đăng ký thành công. Tại đây, sinh viên có thể theo dõi thời gian đăng ký cụ thể và thực hiện thao tác “Hủy đăng ký” trực tuyến đối với từng học phần khi cần thiết.



Hình 5.4: Giao diện trang quản lý môn học và chức năng hủy học phần

5.2.4.5 Kết quả test

Đã viết 2 test case cho CancellationService bằng JUnit:

STT	Tên test case	Input	Kết quả mong đợi
1	testCancelRegistration_ThanhCong	studentId="SV001", courseId="HP001"	Hủy thành công, chi tiết đăng ký giảm còn 0 phần tử
2	testCancelRegistration_MonChua DangKy_BaoLoi	studentId="SV001",cou rseId="HP999"	RegistrationNotFo undException

Bảng 5.3 Bảng kiểm thử module hủy đăng ký và tổng hợp kết quả

Kết quả chạy mvn test: **Tests run: 2, Failures: 0, Errors: 0 — BUILD SUCCESS.**

5.2.5 Module ghi nhật ký hệ thống

5.2.5.1 Mô tả chức năng

Module Ghi nhật ký hệ thống chịu trách nhiệm tự động theo dõi, lưu vết toàn bộ các hoạt động và giao dịch quan trọng diễn ra trong ứng dụng. Khi sinh viên thực hiện các thao tác như đăng nhập hệ thống, đăng ký thành công một học phần hay thực hiện hủy đăng ký môn học, module sẽ tự động ghi nhận lại thông tin chi tiết. Chức năng này đóng vai trò cốt lõi trong việc đảm bảo tính minh bạch, hỗ trợ công tác kiểm tra, truy xuất lịch sử thao tác và xử lý sự cố khi cần thiết.

5.2.5.2 Các class/method chính

Lớp thực thể LogEntry:

- Đại diện cho một bản ghi nhật ký đơn lẻ, lưu trữ các trường dữ liệu chi tiết bao gồm: mã định danh bản ghi (id), mốc thời gian diễn ra sự kiện (timestamp), đối tượng thực hiện (actor), hành động cụ thể (action) và kết quả thực thi (result).
- Phương thức toFormattedString(): Chuyển đổi thông tin nhật ký thành chuỗi định dạng chuẩn để thuận tiện cho việc ghi file.

Lớp LogRepository: Đảm nhiệm tầng truy xuất dữ liệu, chịu trách nhiệm đọc và ghi các đối tượng LogEntry trực tiếp xuống tệp lưu trữ giả lập logs.json.

Lớp LogService: Cung cấp phương thức công khai log(String action, String actor, String result). Phương thức này được tích hợp (tiêm phụ thuộc) vào các service nghiệp vụ khác như RegistrationService hay AuthService để tự động hóa hoàn toàn quá trình ghi vết ngay khi giao dịch hoàn tất.

5.2.5.3 Nguyên lý OOP áp dụng trong module này

Separation of Concerns (Phân tách mối quan tâm): Module hoạt động hoàn toàn độc lập, tách biệt rõ ràng giữa việc xử lý nghiệp vụ chính và việc lưu trữ lịch sử, giúp mã nguồn mạch lạc và dễ bảo trì.

Đóng gói (Encapsulation): Các thuộc tính bên trong lớp LogEntry được khai báo ở mức private và tuân thủ nguyên tắc truy xuất dữ liệu thông qua các phương thức getter/setter.

5.2.5.4 Ảnh minh họa

Minh họa kết quả thực thi tự động của LogService và LogRepository, ghi nhận lại chi tiết mốc thời gian, mã định dạng người dùng (USER), vai trò (ROLE), hành động thực hiện (ACTION), trạng thái thành công (STATUS) và nội dung chi tiết (DETAIL).

```
[2026-08-06 02:02:19]
USER: 24102222
ROLE: Student
ACTION: REGISTER_COURSE
STATUS: SUCCESS
DETAIL: Đăng ký môn FFS703007-01.
---
[2026-08-06 02:02:28]
USER: 24102222
ROLE: Student
ACTION: REGISTER_COURSE
STATUS: SUCCESS
DETAIL: Đăng ký môn CSE703048-01.
```

Hình 5.5 Minh họa định dạng bản ghi nhật ký hệ thống khi sinh viên thao tác đăng ký học phần

5.2.5.5 Kết quả test

Quá trình kiểm thử module Ghi nhật ký hệ thống được thực hiện thông qua cơ chế tích hợp ngầm (integration logging) trong các luồng nghiệp vụ chính (như đăng ký và hủy môn học):

STT	Tên test case	Luồng thực thi trong code	Kết quả thực tế
1	Kiểm tra ghi log khi thao tác nghiệp vụ.	RegistrationService / CancellationService gọi phương thức LogService.log() sau khi giao dịch thành công.	Bản ghi nhật ký được tự động khởi tạo và ghi xuống tệp logs.json với đầy đủ thông tin thời gian, tác nhân và trạng thái.

2	Kiểm tra tính độc lập của Log.	LogService thực hiện ghi file bằng LogRepository tách biệt hoàn toàn với luồng xử lý chính.	Đảm bảo nếu có phát sinh thao tác ghi log, hệ thống vẫn duy trì tốc độ xử lý nhanh và không làm gián đoạn luồng dữ liệu của sinh viên.
---	--------------------------------	---	--

Bảng 5.4 Bảng kiểm thử module ghi nhật ký hệ thống

Kết quả thực thi toàn hệ thống: Các thao tác kiểm thử tích hợp trên các module liên quan đều hoạt động ổn định, ghi nhận dữ liệu nhật ký chính xác vào file logs.json và đạt trạng thái **BUILD SUCCESS**.

5.3 Tiện ích và Xử lý ngoại lệ

Để đảm bảo mã nguồn gọn gàng và xử lý các kịch bản lỗi thân thiện với người dùng, hệ thống được cấu trúc thêm hai thành phần hỗ trợ quan trọng:

- Tổ chức lớp tiện ích (Utilities): Các đoạn mã có tính chất dùng chung được bóc tách và đóng gói vào thư mục utils/ dưới dạng các lớp tĩnh (static utility classes). Cụ thể: FileUtils hỗ trợ chuẩn hóa việc đọc/ghi dữ liệu với định dạng JSON thông qua thư viện Gson; IdGenerator chịu trách nhiệm sinh các chuỗi mã định danh (ID) tự động và duy nhất cho các bản ghi; PasswordUtils cung cấp thuật toán băm (hash) để bảo mật mật khẩu người dùng trước khi lưu trữ.
- Xử lý ngoại lệ hướng đối tượng (Custom Exceptions): Thay vì sử dụng các lỗi hệ thống khô khan khó kiểm soát, dự án đã xây dựng một cây hình tháp ngoại lệ tùy chỉnh tại package exception/. Các lớp ngoại lệ đặc thù như UserNotFoundException, CourseNotFoundException, CourseFullException, DuplicateRegistrationException, CreditLimitExceededException và ScheduleConflictException được ném ra từ tầng Service. Tầng Handler sẽ chịu trách nhiệm bắt (catch) các ngoại lệ này và chuyển đổi thành HTTP Response mang mã lỗi 400 (Bad Request) kèm theo thông báo tiếng Việt rõ ràng, tường minh trả về cho tầng Frontend hiển thị cho sinh viên.

CHƯƠNG 6: KIỂM THỬ HỆ THỐNG

Sau khi hoàn thành việc phân tích, thiết kế và hiện thực các module chức năng, nhóm tiến hành kiểm thử toàn bộ hệ thống nhằm đánh giá mức độ đáp ứng các yêu cầu nghiệp vụ đã đề ra. Khác với các ca kiểm thử đơn vị được trình bày trong Chương 5, chương này tập trung vào việc kiểm tra hoạt động của hệ thống như một chỉnh thể, đánh giá khả năng phối hợp giữa các module cũng như mức độ ổn định trong các tình huống sử dụng thực tế.

Mục tiêu của quá trình kiểm thử bao gồm:

- Xác nhận các chức năng nghiệp vụ hoạt động đúng theo yêu cầu phân tích.
- Đánh giá tính nhất quán của dữ liệu trong quá trình xử lý.
- Kiểm tra khả năng xử lý các tình huống bất thường và ngoại lệ.
- Đảm bảo các quy tắc đăng ký môn học luôn được thực thi trong mọi trường hợp.
- Đánh giá khả năng phối hợp giữa các tầng Presentation, Service và Repository của hệ thống.

Quá trình kiểm thử được thực hiện trên môi trường phát triển sử dụng Java 17, Spring Boot, Maven, JUnit 5 và cơ chế lưu trữ dữ liệu bằng các tệp JSON. Các thao tác kiểm thử được tiến hành trực tiếp trên những chức năng mà người sử dụng thực hiện thông qua giao diện và các REST API của hệ thống.

6.1 Môi trường và phương pháp kiểm thử

Để đảm bảo kết quả phản ánh đúng quá trình vận hành của hệ thống, nhóm thực hiện kiểm thử trên cùng môi trường được sử dụng trong quá trình thực hiện hệ thống.

Môi trường kiểm thử:

Thành phần	Công nghệ sử dụng
Ngôn ngữ	Java 17
Framework Backend	Spring Boot
Frontend	ReactJS + Vite
Build Tool	Maven
Công cụ kiểm thử	JUnit 5
Dữ liệu kiểm thử	Mock Database (JSON)
IDE	IntelliJ IDEA

Các trường hợp kiểm thử được xây dựng theo hướng mô phỏng hành vi thực tế của người dùng. Thay vì kiểm tra từng phương thức riêng lẻ, hệ thống được đánh giá dựa trên toàn bộ luồng nghiệp vụ từ khi người dùng gửi yêu cầu đến khi dữ liệu được cập nhật vào kho lưu trữ.

Quá trình kiểm thử bao gồm 2 nhóm chính:

1. Kiểm thử chức năng (Functional Testing): xác minh từng chức năng hoạt động theo đúng yêu cầu.
2. Kiểm thử ngoại lệ (Exception Testing): đánh giá khả năng xử lý các trường hợp không ngoại lệ nhằm đảm bảo hệ thống luôn duy trì trạng thái ổn định

6.2 Kiểm thử các nghiệp vụ chính

6.2.1 Kiểm thử chức năng đăng nhập

Chức năng đăng nhập là bước đầu tiên trước khi sinh viên sử dụng hệ thống. Quá trình kiểm thử tập trung đánh giá khả năng xác thực tài khoản và phản hồi của hệ thống đối với các trường hợp hợp lệ và không hợp lệ.

Test ID	Nội dung kiểm thử	Kết quả mong đợi	Trạng thái
TC-01	Đăng nhập đúng tài khoản	Đăng nhập thành công	Pass
TC-02	Sai mật khẩu	Hiện thị thông báo lỗi	Pass
TC-03	Tài khoản không tồn tại	Không cho phép đăng nhập	Pass

Kết quả cho thấy cơ chế xác thực hoạt động đúng, chỉ cho phép người dùng hợp lệ truy cập hệ thống.

6.2.2 Kiểm thử chức năng tra cứu môn học

Nhóm tiến hành kiểm tra khả năng tải danh sách học phần, tìm kiếm theo mã môn và theo tên môn.

Test ID	Nội dung kiểm thử	Kết quả mong đợi	Trạng thái
---------	-------------------	------------------	------------

TC-04	Hiển thị toàn bộ môn học	Danh sách đầy đủ	Pass
TC-05	Tìm theo mã môn	Trả về đúng học phần	Pass
TC-06	Tìm theo tên môn	Hiển thị chính xác kết quả	Pass
TC-07	Tìm môn không tồn tại	Danh sách rỗng	Pass

Qua các lần kiểm thử, dữ liệu được đọc chính xác từ tệp JSON và phản hồi đúng với từng điều kiện tìm kiếm.

6.2.3 Kiểm thử nghiệp vụ đăng ký môn học

Đây là chức năng quan trọng nhất của hệ thống nên được kiểm thử với nhiều tình huống khác nhau nhằm đảm bảo mọi quy tắc nghiệp vụ đều được thực thi.

Test ID	Kịch bản	Kết quả mong đợi	Trạng thái
TC-08	Đăng ký môn hợp lệ	Đăng ký thành công	Pass
TC-09	Vượt giới hạn tín chỉ	Từ chối đăng ký	Pass
TC-10	Trùng lịch học	Từ chối đăng ký	Pass
TC-11	Lớp học đã đầy	Từ chối đăng ký	Pass
TC-12	Đăng ký trùng môn	Từ chối đăng ký	Pass

Kết quả kiểm thử chứng minh bộ Validator của hệ thống hoạt động chính xác. Mỗi điều kiện được kiểm tra độc lập trước khi dữ liệu được ghi vào hệ thống, nhờ đó không phát sinh các trường hợp đăng ký sai quy định.

6.2.4 Kiểm thử chức năng huỷ đăng ký

Nhóm tiến hành kiểm thử chức năng huỷ học phần nhằm đảm bảo dữ liệu luôn được cập nhật chính xác sau khi sinh viên thay đổi kế hoạch học tập.

Test ID	Nội dung kiểm thử	Kết quả mong đợi	Trạng thái
TC-13	Huỷ môn đã đăng ký	Huỷ thành công	Pass

TC-14	Hủy môn chưa đăng ký	Báo lỗi	Pass
TC-15	Cập nhật số chỗ lớp học	Chính xác	Pass

Sau khi hủy môn học, số lượng sinh viên của lớp được cập nhật đồng thời thông tin đăng ký của sinh viên cũng được đồng bộ trong kho dữ liệu

6.2.5 Kiểm thử chức năng tổng hợp kết quả đăng ký

Sau mỗi lần đăng ký hoặc hủy học phần, hệ thống cần tính toán lại tổng số tín chỉ và hiển thị danh sách môn học hiện tại của sinh viên.

Test ID	Nội dung kiểm thử	Kết quả mong đợi	Trạng thái
TC-16	Tính tổng tín chỉ	Chính xác	Pass
TC-17	Hiển thị danh sách môn đã đăng ký	Đầy đủ	Pass
TC-18	Sau khi hủy môn	Dữ liệu cập nhật đúng	Pass

Các kết quả cho thấy hệ thống luôn đồng bộ giữa dữ liệu đăng ký và kết quả hiển thị trên giao diện.

6.2.6 Kiểm thử chức năng ghi nhật ký hệ thống

Mọi thao tác quan trọng của người dùng đều được lưu lại nhằm phục vụ công tác theo dõi và truy vết.

Test ID	Nội dung kiểm thử	Kết quả mong đợi	Trạng thái
TC-19	Ghi log đăng nhập	Thành công	Pass
TC-20	Ghi log đăng ký môn	Thành công	Pass
TC-21	Ghi log hủy môn	Thành công	Pass
TC-22	Ghi nhiều log liên tiếp	Không mất dữ liệu	Pass

Thông qua kiểm thử có thể thấy cơ chế ghi nhật ký hoạt động ổn định và lưu trữ đầy đủ lịch sử thao tác của người dùng.

6.3 Đánh giá kết quả kiểm thử

Kết quả kiểm thử cho thấy toàn bộ các chức năng cốt lõi của hệ thống đều hoạt động đúng theo yêu cầu đã được phân tích ở giai đoạn thiết kế. Các quy tắc nghiệp vụ như giới hạn số tín chỉ, kiểm tra sĩ số lớp học, ngăn chặn

đăng ký trùng lặp, phát hiện xung đột thời khóa biểu và kiểm soát việc hủy đăng ký đều được thực thi chính xác.

Trong quá trình kiểm thử, dữ liệu được cập nhật đồng bộ giữa các thành phần của hệ thống. Sau mỗi thao tác đăng ký hoặc hủy môn học, các tệp dữ liệu JSON phản ánh đúng trạng thái hiện tại của hệ thống, đồng thời thông tin tổng hợp về số tín chỉ và danh sách học phần của sinh viên cũng được cập nhật ngay lập tức. Điều này cho thấy sự phối hợp hiệu quả giữa các tầng Repository, Service và Handler trong kiến trúc ba tầng mà nhóm đã xây dựng.

Bên cạnh các trường hợp thành công, nhóm cũng kiểm thử nhiều tình huống ngoại lệ nhằm đánh giá khả năng chịu lỗi của hệ thống. Các yêu cầu không hợp lệ đều được phát hiện và xử lý thông qua các lớp ngoại lệ chuyên biệt, giúp ngăn chặn việc ghi dữ liệu sai và trả về thông báo rõ ràng cho người sử dụng. Cơ chế này góp phần nâng cao tính ổn định và độ tin cậy của phần mềm.

Tổng cộng hệ thống đã được thực hiện 22 kịch bản kiểm thử, bao phủ hầu hết các chức năng chính của đề tài. Kết quả cho thấy tất cả các kịch bản đều đạt yêu cầu, không phát hiện lỗi nghiêm trọng ảnh hưởng đến hoạt động của hệ thống.

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Kết luận

Sau thời gian nghiên cứu, lập kế hoạch và triển khai thực tế, hệ thống đăng ký môn học cho sinh viên đã hoàn hành đúng tiến độ và đạt được các mục tiêu đề ra.

Những kết quả chính mà nhóm đã đạt được bao gồm:

- ✓ Chuẩn hóa quy trình nghiệp vụ: Xây dựng thành công hệ thống mô phỏng công thông tin đào tạo đại học, đáp ứng trọn vẹn các tính năng cốt lõi như quản lý danh mục học phần, xây dựng thời khóa biểu cá nhân, cơ chế đăng ký/hủy môn học linh hoạt và hệ thống ghi nhật ký hệ thống (LogService) minh bạch.
- ✓ Ứng dụng mô hình thiết kế phần mềm: Áp dụng thành công các nguyên lý lập trình hướng đối tượng (OOP) như tính kế thừa, đa hình, đóng gói, cùng các Design Pattern tiêu biểu (đặc biệt là Strategy Pattern trong module kiểm tra điều kiện đăng ký), giúp mã nguồn có tính mô-đun cao, dễ dàng bảo trì và mở rộng.
- ✓ Đảm bảo chất lượng mã nguồn: Xây dựng hệ thống kiểm thử tự động (Unit Test / Integration Test) cho các module quan trọng, đảm bảo các ràng buộc nghiệp vụ (giới hạn tín chỉ, sĩ số lớp, trùng lịch học) hoạt động chính xác và đạt trạng thái phản hồi BUILD SUCCESS.

7.2 Đánh giá kết quả đạt được

Mặc dù đã hoàn thiện các chức năng cơ bản, nhóm nhận thấy dự án vẫn còn một số điểm hạn chế nhất định do giới hạn về thời gian và quy mô nghiên cứu:

1. Hệ thống hiện tại lưu trữ dữ liệu chủ yếu qua tệp (JSON) và cơ sở dữ liệu giả lập, chưa kết nối với hệ quản trị cơ sở dữ liệu quan hệ lớn (như MySQL hay PostgreSQL) phục vụ cho môi trường thực tế có tải trọng cao.
2. Giao diện người dùng (Frontend) mới tập trung vào trải nghiệm cốt lõi của sinh viên, chưa xây dựng đầy đủ phân quyền chi tiết cho các cấp quản trị viên khác như Giảng viên hay Phòng Đào tạo.

7.3 Hướng phát triển trong tương lai

Để hệ thống ngày càng hoàn thiện và có khả năng ứng dụng vào thực tiễn cao hơn, nhóm định hướng các bước phát triển tiếp theo:

- **Nâng cấp cơ sở dữ liệu:** Chuyển đổi toàn bộ hệ thống lưu trữ sang cơ sở dữ liệu quan hệ có tối ưu hóa chỉ mục (Index) để tăng tốc độ truy vấn khi số lượng sinh viên đăng ký đồng thời lớn.
- **Mở rộng phân quyền (RBAC):** Xây dựng thêm phân hệ dành riêng cho Giảng viên (nhập điểm, điểm danh) và Phòng Đào tạo (mở kỳ học, quản lý chương trình đào tạo).
- **Triển khai thực tế:** Đưa ứng dụng lên môi trường điện toán đám mây (Cloud Server) để thử nghiệm khả năng chịu tải thực tế trong các đợt đăng ký tín chỉ cao điểm của trường.

TÀI LIỆU THAM KHẢO

1. **Nguyễn Văn Ba** (2020), *Giáo trình Lập trình Hướng đối tượng nâng cao*, Nhà xuất bản Đại học Quốc gia Hà Nội.
2. **Nguyễn Thanh Hùng** (2022), *Phân tích và Thiết kế Hệ thống Thông tin*, Nhà xuất bản Khoa học và Kỹ thuật.
3. **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994), *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional.
4. **Walls, C.** (2022), *Spring Boot in Action* (2nd ed.), Manning Publications.
5. **Fowler, M.** (2002), *Patterns of Enterprise Application Architecture*, Addison-Wesley.
6. **Oracle** (2023), *Java SE Documentation & API Specification*.